

a_tensor_initialization.py

```
In [1]: import torch
```

```
In [2]: # torch.Tensor class 생성자를 통한 텐서 생성
t1 = torch.Tensor([1, 2, 3], device='cpu') #! 기본 floating dtype으로 생성됨
print(t1.dtype) # >>> torch.float32
print(t1.device) # >>> cpu
print(t1.requires_grad) # >>> False
print(t1.size()) # torch.Size([3])
print(t1.shape) # torch.Size([3])
```

```
torch.float32
cpu
False
torch.Size([3])
torch.Size([3])
```

```
In [3]: #! gpu device가 있다면 아래와 같이 GPU로 옮길 수 있음
# t1_cuda = t1.to(torch.device('cuda'))
# t1_cuda = t1.cuda()
#! CPU로 옮기기
t1_cpu = t1.cpu()
```

```
In [4]: #! [추가 코드] CUDA 사용 가능 여부에 따라 GPU 또는 CPU 선택
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
t1 = t1.to(device)
print(t1.device) # >>> cuda:0 (GPU가 있다면) / cpu (GPU가 없다면)
```

```
cpu
```

```
In [5]: # torch.tensor function을 통한 텐서 생성
t2 = torch.tensor([1, 2, 3], device='cpu') #! 동적으로 타입이 결정됨
print(t2.dtype) # >>> torch.int64
print(t2.device) # >>> cpu
print(t2.requires_grad) # >>> False
print(t2.size()) # torch.Size([3])
print(t2.shape) # torch.Size([3])
```

```
torch.int64
cpu
False
torch.Size([3])
torch.Size([3])
```

```
In [6]: #! gpu device가 있다면 아래와 같이 GPU로 옮길 수 있음
# t2_cuda = t2.to(torch.device('cuda'))
# t2_cuda = t2.cuda()
#! cpu로 옮기기
t2_cpu = t2.cpu()
```

```
In [7]: #! 0차원 텐서(스칼라) 생성
a1 = torch.tensor(1) # shape: torch.Size([]), ndims(=rank): 0
print(a1.shape, a1.ndim)
```

```
torch.Size([]) 0
```

```
In [8]: #! 1차원 텐서(벡터) 생성
a2 = torch.tensor([1]) # shape: torch.Size([1]), ndims(=rank): 1
print(a2.shape, a2.ndim)
```

```
torch.Size([1]) 1
```

```
In [9]: #! 크기가 n인 벡터 생성
a3 = torch.tensor([1, 2, 3, 4, 5]) # shape: torch.Size([5]), ndims(=rank): 1
print(a3.shape, a3.ndim)
```

```
torch.Size([5]) 1
```

```
In [10]: #! 2차원 텐서(행렬) 생성
a4 = torch.tensor([[1], [2], [3], [4], [5]]) # shape: torch.Size([5, 1]), ndims(=rank): 2
print(a4.shape, a4.ndim)
```

```
torch.Size([5, 1]) 2
```

```
In [11]: a5 = torch.tensor([ # shape: torch.Size([3, 2]), ndims(=rank): 2
    [1, 2],
    [3, 4],
    [5, 6]
])
print(a5.shape, a5.ndim)
```

```
torch.Size([3, 2]) 2
```

```
In [12]: a6 = torch.tensor([                # shape: torch.Size([3, 2, 1]), ndims(=rank): 3
    [[1], [2]],
    [[3], [4]],
    [[5], [6]]
])
print(a6.shape, a6.ndim)
```

```
torch.Size([3, 2, 1]) 3
```

```
In [13]: a7 = torch.tensor([                # shape: torch.Size([3, 1, 2, 1]), ndims(=rank): 4
    [[[1], [2]]],
    [[[3], [4]]],
    [[[5], [6]]]
])
print(a7.shape, a7.ndim)
```

```
torch.Size([3, 1, 2, 1]) 4
```

```
In [14]: a8 = torch.tensor([                # shape: torch.Size([3, 1, 2, 3]), ndims(=rank): 4
    [[[1, 2, 3], [2, 3, 4]]],
    [[[3, 1, 1], [4, 4, 5]]],
    [[[5, 6, 2], [6, 3, 1]]]
])
print(a8.shape, a8.ndim)
```

```
torch.Size([3, 1, 2, 3]) 4
```

```
In [15]: a9 = torch.tensor([                # shape: torch.Size([3, 1, 2, 3, 1]), ndims(=rank): 5
    [[[[1], [2], [3]], [[2], [3], [4]]]],
    [[[[3], [1], [1]], [[4], [4], [5]]]],
    [[[[5], [6], [2]], [[6], [3], [1]]]]
])
print(a9.shape, a9.ndim)
```

```
torch.Size([3, 1, 2, 3, 1]) 5
```

```
In [16]: a10 = torch.tensor([                # shape: torch.Size([4, 5]), ndims(=rank): 2
    [1, 2, 3, 4, 5],
    [1, 2, 3, 4, 5],
    [1, 2, 3, 4, 5],
    [1, 2, 3, 4, 5],
])
print(a10.shape, a10.ndim)
```

```
torch.Size([4, 5]) 2
```

```
In [17]: a10 = torch.tensor([                # shape: torch.Size([4, 1, 5]), ndims(=rank): 3
    [[1, 2, 3, 4, 5]],
    [[1, 2, 3, 4, 5]],
    [[1, 2, 3, 4, 5]],
    [[1, 2, 3, 4, 5]],
])
print(a10.shape, a10.ndim)
```

```
torch.Size([4, 1, 5]) 3
```

```
In [18]: #! 내부 리스트의 길이가 달라 Tensor를 만들 수 없음
try:
    a11 = torch.tensor([
        [[1, 2, 3], [4, 5]],
        [[1, 2, 3], [4, 5]],
        [[1, 2, 3], [4, 5]],
        [[1, 2, 3], [4, 5]]
    ])
except ValueError as error:
    print("내부 리스트의 길이가 달라 Tensor를 만들 수 없음")
    print(type(error).__name__, error)
```

내부 리스트의 길이가 달라 Tensor를 만들 수 없음

ValueError expected sequence of length 3 at dim 3 (got 2)

기술적 사항 및 고찰

- `torch.tensor()` 혹은 `torch.Tensor()` 로 tensor를 생성할 수 있음
- `torch.tensor()` 는 입력값을 보고 dtype을 추론하지만 `torch.Tensor()` 는 기본적으로 실수형 tensor를 생성함
- tensor의 shape은 중첩 리스트의 각 단계에 의해 결정됨

b_tensor_initialization_copy.py

```
In [19]: import torch
```

```
import numpy as np
```

```
In [20]: #! list를 Tensor로 복사  
#! 원본과 메모리를 공유하지 않음  
l1 = [1, 2, 3]  
t1 = torch.Tensor(l1)  
  
l2 = [1, 2, 3]  
t2 = torch.tensor(l2)  
  
l3 = [1, 2, 3]  
t3 = torch.as_tensor(l3)  
  
l1[0] = 100  
l2[0] = 100  
l3[0] = 100  
  
print(t1)  
print(t2)  
print(t3)
```

```
tensor([1., 2., 3.])  
tensor([1, 2, 3])  
tensor([1, 2, 3])
```

```
In [21]: #! NumPy 배열을 Tensor로 변환했을 때의 메모리 공유 여부 비교  
l4 = np.array([1, 2, 3])  
t4 = torch.Tensor(l4)  
  
l5 = np.array([1, 2, 3])  
t5 = torch.tensor(l5)  
  
l6 = np.array([1, 2, 3])  
t6 = torch.as_tensor(l6)    #! np.ndarray를 torch.as_tensor()로 변환할 때 메모리가 공유됨  
  
l4[0] = 100  
l5[0] = 100  
l6[0] = 100  
  
print(t4)  
print(t5)  
print(t6)
```

```
tensor([1., 2., 3.])  
tensor([1, 2, 3])  
tensor([100,  2,  3])
```

기술적 사항 및 고찰

- Python `list` 로 Tensor를 만들면 세 생성 방식 모두 원본 `list`와 독립된 메모리를 사용함
- `torch.tensor()` 는 입력 데이터를 복사하지만, `torch.as_tensor()` 는 가능한 경우 기존 메모리를 공유함
- NumPy 배열에 `torch.as_tensor()` 를 적용한 경우 원본 배열의 변경이 Tensor에도 반영되는 것으로 메모리 공유를 확인할 수 있었음

c_tensor_initialization_with_size.py

```
In [22]: import torch
```

```
In [23]: #! torch.ones(): 모든 원소가 1인 tensor를 생성함  
t1 = torch.ones(size=(5,)) # or torch.ones(5)  
print(t1)  
  
#! torch.ones_like(): input tensor와 shape, dtype이 같은 ones tensor를 생성함  
t1_like = torch.ones_like(input=t1)  
print(t1_like)
```

```
tensor([1., 1., 1., 1., 1.])  
tensor([1., 1., 1., 1., 1.])
```

```
In [24]: #! torch.zeros(): 모든 원소가 0인 tensor를 생성함  
t2 = torch.zeros(size=(6,)) # or torch.zeros(6)  
print(t2)  
  
#! torch.zeros_like(): input tensor와 shape, dtype이 같은 zeros tensor를 생성함  
t2_like = torch.zeros_like(input=t2)  
print(t2_like)
```

```
tensor([0., 0., 0., 0., 0., 0.])  
tensor([0., 0., 0., 0., 0., 0.])
```

```
In [25]: #! torch.empty(): 메모리 공간만 확보하고 값을 초기화하지 않으므로 실행할 때마다 임의의 값이 나타날 수 있음  
#! 메모리 공간만 확보하므로 이전 메모리 값이 나타날 수 있음
```

```

#! 현재 출력이 0이어도 항상 0이 나온다고 보장되지 않음
t3 = torch.empty(size=(4,)) # or torch.empty(4)
print(t3)

#! torch.empty_like(): input tensor와 shape, dtype이 같은 empty tensor를 생성함
t3_like = torch.empty_like(input=t3)
print(t3_like)

```

```

tensor([0., 0., 0., 0.])
tensor([0., 0., 0., 0.])

```

```

In [26]: #! torch.eye(n): n×n 단위행렬 생성
t4 = torch.eye(n=3)
print(t4)

```

```

tensor([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.]])

```

```

In [27]: #! [추가 코드] 단위행렬은 one-hot encoding에 활용 가능
labels = torch.tensor([1, 2, 0]) # (3, )
eye = torch.eye(3)
one_hot_labels = eye[labels] # (3, 3)

print(one_hot_labels)

```

```

tensor([[0., 1., 0.],
        [0., 0., 1.],
        [1., 0., 0.]])

```

기술적 사항 및 고찰

- `ones()`, `zeros()`, `empty()` 는 지정한 shape으로 Tensor를 생성하고, `eye()` 는 단위행렬을 생성함
- 이름에 `_like` 가 붙은 함수는 입력 Tensor의 shape, dtype, device를 기준으로 새로운 Tensor를 생성함
- `empty()` 는 값을 0으로 채우지 않으므로 출력값을 데이터로 사용하기 전에 반드시 초기화해야 함

d_tensor_initialization_random_values.py

```

In [28]: import torch

```

```

In [29]: #! torch.randint(): 정수 난수 생성
t1 = torch.randint(low=10, high=20, size=(1, 2)) #! low 이상 high 미만의 정수 난수 생성
print(t1)

```

```

tensor([[19, 14]])

```

```

In [30]: #! torch.rand(): 균등분포 샘플링
t2 = torch.rand(size=(1, 3)) #!  $x \sim U(0, 1)$ 
print(t2)

```

```

tensor([[0.9181, 0.7728, 0.3222]])

```

```

In [31]: #! torch.randn(): 표준정규분포 샘플링
t3 = torch.randn(size=(1, 3)) #!  $x \sim N(0, 1)$ 
print(t3)

```

```

tensor([[ -0.8677,  0.1572,  0.4704]])

```

```

In [32]: #! torch.normal(): 평균 mean, 표준편차 std인 정규분포 샘플링
t4 = torch.normal(mean=10.0, std=1.0, size=(3, 2)) #!  $x \sim N(10.0, 1.0)$ 
print(t4)

```

```

tensor([[11.1117,  9.4460],
        [ 8.9867,  9.2478],
        [11.4790,  9.2811]])

```

```

In [33]: #! torch.linspace(): start부터 end까지 steps만큼의 element를 등차수열로 생성 (1차원 텐서)
t5 = torch.linspace(start=0.0, end=5.0, steps=7)
print(t5)

```

```

tensor([0.0000, 0.8333, 1.6667, 2.5000, 3.3333, 4.1667, 5.0000])

```

```

In [34]: #! torch.arange(): start 이상 end 미만의 수열 생성
t6 = torch.arange(5)
print(t6, t6.dtype)

```

```

#! [추가 코드] 실수 수열도 생성 가능
t6 = torch.arange(5.)
print(t6, t6.dtype)

```

```

tensor([0, 1, 2, 3, 4]) torch.int64
tensor([0., 1., 2., 3., 4.]) torch.float32

```

```
In [35]: #! torch.manual_seed(): 난수 생성 순서 고정
torch.manual_seed(1729)
random1 = torch.rand(2, 3)
random2 = torch.rand(2, 3)
print(random1)
print(random2)

tensor([[0.3126, 0.3791, 0.3087],
        [0.0736, 0.4216, 0.0691]])
tensor([[0.2332, 0.4047, 0.2162],
        [0.9927, 0.4128, 0.5938]])
```

```
In [36]: #! 위의 난수 생성 순서를 재현할 수 있음
torch.manual_seed(1729)
random3 = torch.rand(2, 3)
random4 = torch.rand(2, 3)
print(random3)
print(random4)

tensor([[0.3126, 0.3791, 0.3087],
        [0.0736, 0.4216, 0.0691]])
tensor([[0.2332, 0.4047, 0.2162],
        [0.9927, 0.4128, 0.5938]])
```

```
In [37]: #! [추가 코드] 학습용 임의 mini-batch와 class label 생성
torch.manual_seed(42)
inputs = torch.randn(32, 784)
labels = torch.randint(0, 10, size=(32,))

print(inputs.shape, inputs.dtype)
print(labels.shape, labels.dtype)

torch.Size([32, 784]) torch.float32
torch.Size([32]) torch.int64
```

기술적 사항 및 고찰

- `randint()` 는 정수 범위, `rand()` 는 0 이상 1 미만의 균등분포, `randn()` 은 표준정규분포에서 값을 샘플링함
- `manual_seed()` 는 난수 하나를 고정하는 것이 아니라 난수 생성기의 시작 상태를 고정하여 같은 난수 순서를 재현함
- 난수의 분포와 관계없이 `size` 인자가 결과 Tensor의 shape을 결정함

e_tensor_type_conversion.py

```
In [38]: import torch
```

```
In [39]: a = torch.ones((2, 3)) #! 기본 dtype은 torch.float32
b = torch.ones((2, 3), dtype=torch.int16)
print(a)
print(b)

tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[1, 1, 1],
        [1, 1, 1]], dtype=torch.int16)
```

```
In [40]: c = torch.rand((2, 3), dtype=torch.float64) * 20. #! 스칼라 값 20.0을 곱하여 0 이상 20 미만의 실수 난수 생성
print(c)

tensor([[11.5115, 7.0453, 18.3587],
        [13.8709, 19.5962, 12.1074]], dtype=torch.float64)
```

```
In [41]: d = b.to(torch.int32) #! torch.tensor().to()는 device 뿐만 아니라 dtype도 바꿀 수 있음
print(d)

tensor([[1, 1, 1],
        [1, 1, 1]], dtype=torch.int32)
```

```
In [42]: double_d = torch.ones(10, 2, dtype=torch.double) #! torch.double은 torch.float64와 동일함
short_e = torch.tensor([[1, 2]], dtype=torch.short) #! torch.short는 torch.int16와 동일함

print(double_d.dtype)
print(short_e.dtype)

torch.float64
torch.int16
```

```
In [43]: #! dtype 변환 방법 1: torch.tensor().double(), torch.tensor().short() 메서드 사용
double_d = torch.zeros(10, 2).double()
short_e = torch.ones(10, 2).short()
```

```
print(double_d.dtype)
print(short_e.dtype)
```

```
torch.float64
torch.int16
```

```
In [44]: #! dtype 변환 방법 2: torch.tensor().to() 메서드 사용
double_d = torch.zeros(10, 2).to(torch.double)
short_e = torch.ones(10, 2).to(dtype=torch.short)

print(double_d.dtype)
print(short_e.dtype)
```

```
torch.float64
torch.int16
```

```
In [45]: #! dtype 변환 방법 3: torch.tensor().type() 메서드 사용
double_d = torch.zeros(10, 2).type(torch.double)
short_e = torch.ones(10, 2).type(dtype=torch.short)

print(double_d.dtype)
print(short_e.dtype)
```

```
torch.float64
torch.int16
```

```
In [46]: double_f = torch.rand(5, dtype=torch.double)      # torch.float64
short_g = double_f.to(torch.short)                        # torch.int16
print((double_f * short_g).dtype)                         #! 곱셈 연산 시 두 텐서의 dtype이 다르면 PyTorch의 타입 승격 규칙을 따라 변환
```

```
torch.float64
```

기술적 사항 및 고찰

- `.double()`, `.short()`, `.to()`, `.type()` 을 이용해 dtype을 바꿀 수 있으며 dtype 변환만으로 shape은 변하지 않음
- 실수 Tensor를 정수 dtype으로 변환하면 반올림하지 않고 소수 부분이 제거됨
- 서로 다른 dtype끼리 연산하면 PyTorch의 타입 승격 규칙에 따라 결과 dtype이 결정됨

f_tensor_operations.py

```
In [47]: import torch
```

```
In [48]: #! 같은 shape과 dtype을 가진 두 Tensor를 생성하여 element-wise 연산에 사용
t1 = torch.ones(size=(2, 3))
t2 = torch.ones(size=(2, 3))

print(t1)
print(t2)
```

```
tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[1., 1., 1.],
        [1., 1., 1.]])
```

```
In [49]: #! t1과 t2의 shape이 같으면 element-wise 연산 수행
#! element-wise 덧셈
t3 = torch.add(t1, t2)
t4 = t1 + t2 #! torch.add()와 동일
print(t3)
print(t4)
```

```
tensor([[2., 2., 2.],
        [2., 2., 2.]])
tensor([[2., 2., 2.],
        [2., 2., 2.]])
```

```
In [50]: #! element-wise 뺄셈
```

```
t5 = torch.sub(t1, t2)
t6 = t1 - t2 #! torch.sub()와 동일
print(t5)
print(t6)
```

```
tensor([[0., 0., 0.],
        [0., 0., 0.]])
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

```
In [51]: #! element-wise 곱셈
```

```
t7 = torch.mul(t1, t2)
t8 = t1 * t2 #! torch.mul()와 동일
```

```
print(t7)
print(t8)
```

```
tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[1., 1., 1.],
        [1., 1., 1.]])
```

```
In [52]: #! element-wise 나눗셈
t9 = torch.div(t1, t2)
t10 = t1 / t2 #! torch.div()와 동일
print(t9)
print(t10)
```

```
tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[1., 1., 1.],
        [1., 1., 1.]])
```

기술적 사항 및 고찰

- `add()`, `sub()`, `mul()`, `div()` 는 같은 위치의 원소끼리 계산하는 element-wise 연산임
- 함수 형태와 `+`, `-`, `*`, `/` 연산자는 각각 같은 계산을 수행함
- 입력 shape이 같으면 결과 shape도 유지되며, shape이 다를 때는 broadcasting 가능 여부를 확인해야 함

g_tensor_operations_mm.py

```
In [53]: import torch
```

```
In [54]: # torch.dot(): 벡터 내적
# (m,) * (m,) → ()
t1 = torch.dot(
    torch.tensor([2, 3]), torch.tensor([2, 1])
)
print(t1, t1.size())
```

```
tensor(7) torch.Size([])
```

```
In [55]: t2 = torch.randn(2, 3)
t3 = torch.randn(3, 2)

# torch.mm(): 행렬곱
# (m, n) * (n, p) → (m, p)
t4 = torch.mm(t2, t3)
print(t2, t2.size())
print(t3, t3.size())
print(t4, t4.size())
```

```
tensor([[ 0.1249, -1.5781, -0.6635],
        [ 0.9366, -0.5759,  1.0020]]) torch.Size([2, 3])
tensor([[[-0.1700,  0.8390],
         [ 0.8995,  0.0264],
         [ 1.0481,  0.5525]]) torch.Size([3, 2])
tensor([[[-2.1361, -0.3034],
         [ 0.3730,  1.3243]]) torch.Size([2, 2])
```

```
In [56]: t5 = torch.randn(10, 3, 4)
t6 = torch.randn(10, 4, 5)

# torch.bmm(): 배치별 행렬곱
# (B, m, n) * (B, n, p) → (B, m, p)
t7 = torch.bmm(t5, t6)
print(t7.size())
```

```
torch.Size([10, 3, 5])
```

기술적 사항 및 고찰

- `dot()` 은 같은 길이의 1차원 Tensor 두 개를 내적하여 0차원 Tensor를 반환함
- `mm()` 은 2차원 행렬곱이고 $(m, n) \times (n, p)$ 의 결과 shape은 (m, p) 임
- `bmm()` 은 3차원 Tensor에서 같은 batch 위치의 행렬끼리 곱하며 두 입력의 batch 크기가 같아야 함

h_tensor_operations_matmul.py

```
In [57]: import torch
```

```
In [58]: #! torch.matmul(): 범용 행렬곱 및 배치 브로드캐스팅
# 벡터 × 벡터
t1 = torch.randn(3) # (m,)
```

```
t2 = torch.randn(3) # (m,)
print(torch.matmul(t1, t2).size()) # (m,) * (m,) → ()
```

torch.Size([])

```
In [59]: # 행렬 × 벡터
t3 = torch.randn(3, 4) # (m, n)
t4 = torch.randn(4)    # (n,)
print(torch.matmul(t3, t4)) # (m, n) * (n,) → (m,)
```

tensor([-2.2168, 0.8540, 0.9579])

```
In [60]: # 배치 행렬 × 벡터
t5 = torch.randn(10, 3, 4) # (B, m, n)
t6 = torch.randn(4)        # (n,)
print(torch.matmul(t5, t6).size()) # (B, m, n) * (n,) → (B, m)
```

torch.Size([10, 3])

```
In [61]: # 배치 행렬 × 배치 행렬: bmm
t7 = torch.randn(10, 3, 4) # (B, m, n)
t8 = torch.randn(10, 4, 5) # (B, n, p)
print(torch.matmul(t7, t8).size()) # (B, m, n) * (B, n, p) → (B, m, p)
```

torch.Size([10, 3, 5])

```
In [62]: # 배치 행렬 × 행렬: 배치 차원에 broadcasting
t9 = torch.randn(10, 3, 4) # (B, m, n)
t10 = torch.randn(4, 5)    # (n, p)
print(torch.matmul(t9, t10).size()) # (B, m, n) * (n, p) → (B, m, p)
```

torch.Size([10, 3, 5])

```
In [63]: #! [추가 코드] 완전연결 계층의 행렬곱 연산
features = torch.randn(32, 128)
weights = torch.randn(128, 10)
bias = torch.randn(10)

logits = torch.matmul(features, weights) + bias
print(logits.shape) # >>> torch.Size([32, 10])
```

torch.Size([32, 10])

```
In [64]: #! [추가 코드] attention score 계산
queries = torch.randn(4, 8, 16) # (batch, sequence, hidden)
keys = torch.randn(4, 8, 16)    # (batch, sequence, hidden)

attention_scores = torch.bmm(queries, keys.transpose(1, 2))
print(attention_scores.shape) # >>> torch.Size([4, 8, 8])
```

torch.Size([4, 8, 8])

기술적 사항 및 고찰

- `matmul()` 은 벡터, 행렬, 배치 행렬을 입력 shape에 맞게 처리하는 범용 행렬곱 함수임
- 2차원 이상의 입력에서는 마지막 두 차원을 행렬 차원으로 사용하고 앞쪽 차원을 batch 차원으로 해석함
- 완전연결 계층의 bias처럼 batch 차원이 없는 Tensor는 행렬곱 결과에 broadcasting되어 더해질 수 있음

i_tensor_broadcasting.py

```
In [65]: import torch
```

```
In [66]: t1 = torch.tensor([1.0, 2.0, 3.0]) #! (3, )
t2 = 2.0 #! 스칼라
print(t1 * t2) #! 스칼라 broadcasting 후 연산
```

tensor([2., 4., 6.])

```
In [67]: t3 = torch.tensor([[0, 1], [2, 4], [10, 10]]) #! (3, 2)
t4 = torch.tensor([4, 5]) #! (2,)
print(t3 - t4) #! t4 dim=0 broadcasting 후 연산
```

tensor([[-4, -4],
 [-2, -1],
 [6, 5]])

```
In [68]: t5 = torch.tensor([[1., 2.], [3., 4.]])
print(t5 + 2.0) # t5.add(2.0)
print(t5 - 2.0) # t5.sub(2.0)
print(t5 * 2.0) # t5.mul(2.0)
print(t5 / 2.0) # t5.div(2.0)
```

```

tensor([[3., 4.],
        [5., 6.]])
tensor([[ -1.,  0.],
        [ 1.,  2.]])
tensor([[2., 4.],
        [6., 8.]])
tensor([[0.5000, 1.0000],
        [1.5000, 2.0000]])
tensor([[ -1.,  0.],
        [ 1.,  2.]])
tensor([[2., 4.],
        [6., 8.]])
tensor([[0.5000, 1.0000],
        [1.5000, 2.0000]])

```

```

In [69]: def normalize(x):
          return x / 255 #! broadcasting

t6 = torch.randn(3, 28, 28)
print(normalize(t6).size())

```

```
torch.Size([3, 28, 28])
```

```

In [70]: t7 = torch.tensor([[1, 2], [0, 3]]) # (2, 2)
          t8 = torch.tensor([[3, 1]])         # (1, 2)
          t9 = torch.tensor([[5], [2]])       # (2, 1)
          t10 = torch.tensor([7])             # (1,)
          print(t7 + t8) # >>> tensor([[4, 3], [3, 4]])
          print(t7 + t9) # >>> tensor([[6, 7], [2, 5]])
          print(t8 + t9) # >>> tensor([[8, 6], [5, 3]])
          print(t7 + t10) # >>> tensor([[ 8, 9], [ 7, 10]])

```

```

tensor([[4, 3],
        [3, 4]])
tensor([[6, 7],
        [2, 5]])
tensor([[8, 6],
        [5, 3]])
tensor([[ 8, 9],
        [ 7, 10]])

```

```

In [71]: t11 = torch.ones(4, 3, 2)
          t12 = t11 * torch.randn(3, 2) #! (1, 3, 2)로 간주하면 dim=1,2의 크기가 같음 -> dim=0 broadcasting
          print(t12.shape)

```

```
torch.Size([4, 3, 2])
```

```

In [72]: t13 = torch.ones(4, 3, 2)
          t14 = t13 * torch.randn(3, 1) #! (1, 3, 1)로 간주하면 dim=1의 크기가 같음 -> dim=0,2 broadcasting
          print(t14.shape)

```

```
torch.Size([4, 3, 2])
```

```

In [73]: t15 = torch.ones(4, 3, 2)
          t16 = t15 * torch.randn(1, 2) #! (1, 1, 2)로 간주하면 dim=2의 크기가 같음 -> dim=0,1 broadcasting
          print(t16.shape)

```

```
torch.Size([4, 3, 2])
```

```

In [74]: t17 = torch.ones(5, 3, 4, 1)
          t18 = torch.randn(3, 1, 1) #! (1, 3, 1, 1)로 간주하면 dim=1,3의 크기가 같음 -> dim=0,2 broadcasting
          print((t17 + t18).size())

```

```
torch.Size([5, 3, 4, 1])
```

```

In [75]: t19 = torch.empty(5, 1, 4, 1) #! dim=1의 크기가 1임 -> dim=1 broadcasting
          t20 = torch.empty(3, 1, 1) #! (1, 3, 1, 1)로 간주하면 dim=3의 크기가 같음 -> dim=0,2 broadcasting
          print((t19 + t20).size())

```

```
torch.Size([5, 3, 4, 1])
```

```

In [76]: t21 = torch.empty(1) #! (1, 1, 1)로 간주하면 dim=1의 크기가 같음 -> dim=0,2 broadcasting
          t22 = torch.empty(3, 1, 7)
          print((t21 + t22).size())

```

```
torch.Size([3, 1, 7])
```

```

In [77]: t23 = torch.ones(3, 3, 3)
          t24 = torch.ones(3, 1, 3) #! dim=0,2의 크기가 같음 -> dim=1 broadcasting
          print((t23 + t24).size())

```

```
torch.Size([3, 3, 3])
```

```

In [78]: #! Broadcasting 불가: 두 텐서의 shape이 호환되지 않음
          try:
            t25 = torch.empty(5, 2, 4, 1)

```

```

t26 = torch.empty(3, 1, 1) #! (1, 3, 1, 1)로 간주하면 dim=1의 크기가 다름 -> RuntimeError
print((t25 + t26).size())
except RuntimeError as error:
    print("Broadcasting 불가: 두 텐서의 shape이 호환되지 않음")
    print(type(error).__name__, error)

```

Broadcasting 불가: 두 텐서의 shape이 호환되지 않음

RuntimeError The size of tensor a (2) must match the size of tensor b (3) at non-singleton dimension 1

```

In [79]: #! 각 원소에 5를 곱함
t27 = torch.ones(4) * 5
print(t27)

```

tensor([5., 5., 5., 5.])

```

In [80]: #! 각 원소를 제곱
t28 = torch.pow(t27, 2)
print(t28)

```

tensor([25., 25., 25., 25.])

```

In [81]: exp = torch.arange(1., 5.)
a = torch.arange(1., 5.)
t29 = torch.pow(a, exp)
print(t29) # >>> tensor([ 1.,  4., 27., 256.])

```

tensor([1., 4., 27., 256.])

```

In [82]: #! [추가 코드] mean과 std의 broadcasting을 이용한 이미지 정규화 예시

images = torch.rand(32, 3, 224, 224)

mean = torch.tensor([0.485, 0.456, 0.406]).reshape(1, 3, 1, 1)
std = torch.tensor([0.229, 0.224, 0.225]).reshape(1, 3, 1, 1)

normalized_images = (images - mean) / std
print(normalized_images.size()) # >>> torch.Size([32, 3, 224, 224])

```

torch.Size([32, 3, 224, 224])

기술적 사항 및 고찰

- Broadcasting은 두 shape을 오른쪽 차원부터 비교하며 크기가 같거나 한쪽이 1일 때 가능함
- 차원 수가 다르면 짧은 shape의 앞쪽에 크기 1인 차원이 있다고 간주함
- Broadcasting은 값을 실제로 여러 번 복사하기보다 하나의 값을 여러 위치에서 사용하는 방식으로 계산함

j_tensor_indexing_slicing.py

```

In [83]: import torch

```

```

In [84]: x = torch.tensor(
    [[0, 1, 2, 3, 4],
     [5, 6, 7, 8, 9],
     [10, 11, 12, 13, 14]]
)

print(x[1]) # >>> tensor([5, 6, 7, 8, 9])
print(x[:, 1]) # >>> tensor([1, 6, 11])
print(x[1, 2]) # >>> tensor(7)
print(x[:, -1]) # >>> tensor([4, 9, 14])

```

tensor([5, 6, 7, 8, 9])

tensor([1, 6, 11])

tensor(7)

tensor([4, 9, 14])

```

In [85]: print(x[1:]) # >>> tensor([[ 5,  6,  7,  8,  9], [10, 11, 12, 13, 14]])
print(x[1:, 3:]) # >>> tensor([[ 8,  9], [13, 14]])

```

tensor([[5, 6, 7, 8, 9],
 [10, 11, 12, 13, 14]])

tensor([[8, 9],
 [13, 14]])

```

In [86]: #! slicing으로 선택한 1~3번 행의 2번 열에 값 1을 대입하며 원본 y가 직접 변경됨
y = torch.zeros((6, 6))
y[1:4, 2] = 1
print(y)
print(y[1:4, 1:4])

```

```

tensor([[0., 0., 0., 0., 0., 0.],
        [0., 0., 1., 0., 0., 0.],
        [0., 0., 1., 0., 0., 0.],
        [0., 0., 1., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0.]])
tensor([[0., 1., 0.],
        [0., 1., 0.],
        [0., 1., 0.]])

```

In [87]: *#! 행과 열 범위를 slicing하며, 범위로 선택한 차원은 결과 Tensor에서도 유지됨*

```

z = torch.tensor(
    [[1, 2, 3, 4],
     [2, 3, 4, 5],
     [5, 6, 7, 8]]
)
print(z[:2])
print(z[1:, 1:3])
print(z[:, 1:])

```

```

tensor([[1, 2, 3, 4],
        [2, 3, 4, 5]])
tensor([[3, 4],
        [6, 7]])
tensor([[2, 3, 4],
        [3, 4, 5],
        [6, 7, 8]])

```

In [88]: *#! 1번 행부터 끝까지, 1번 열 이상 3번 열 미만 영역을 0으로 바꾸어 원본 z를 수정*

```

z[1:, 1:3] = 0
print(z)

```

```

tensor([[1, 2, 3, 4],
        [2, 0, 0, 5],
        [5, 0, 0, 8]])

```

In [89]: *#! [추가 코드] batch와 class 기준으로 logit 선택*

```

logits = torch.randn(32, 10)
first_sample = logits[0]
first_five_samples = logits[:5]
class_zero_scores = logits[:, 0]

print(first_sample.shape)      # >>> torch.Size([10])
print(first_five_samples.shape) # >>> torch.Size([5, 10])
print(class_zero_scores.shape)  # >>> torch.Size([32])

```

```

torch.Size([10])
torch.Size([5, 10])
torch.Size([32])

```

기술적 사항 및 고찰

- 정수 인덱싱은 선택한 차원을 제거할 수 있지만 범위 slicing은 해당 차원을 유지함
- `:` 는 해당 차원의 모든 원소를 선택하고 음수 인덱스는 뒤에서부터 원소를 선택함
- slicing한 영역에 값을 대입하면 원본 Tensor의 해당 위치 값이 직접 변경됨

k_tensor_reshaping.py

In [90]: **import** torch

In [91]: `t1 = torch.tensor([[1, 2, 3], [4, 5, 6]])` # (2, 3)

```

#! view()와 reshape()는 모두 텐서의 shape을 변경
t2 = t1.view(3, 2)      # (2, 3) -> (3, 2)
t3 = t1.reshape(1, 6)   # (2, 3) -> (1, 6)

t2[0, 0] = 100
t3[0, 1] = 200

print(t1)
print(t2)
print(t3)

```

```

tensor([[100, 200,  3],
        [ 4,  5,  6]])
tensor([[100, 200],
        [ 3,  4],
        [ 5,  6]])
tensor([[100, 200,  3,  4,  5,  6]])

```

```
In [92]: t4 = torch.arange(8).view(2, 4) # Shape becomes (2, 4)
t5 = torch.arange(6).view(2, 3) # Shape becomes (2, 3)
print(t4)
print(t5)

tensor([[0, 1, 2, 3],
        [4, 5, 6, 7]])
tensor([[0, 1, 2],
        [3, 4, 5]])
```

```
In [93]: # Original tensor with shape (1, 3, 1)
t6 = torch.tensor([[[1], [2], [3]]])

# 크기가 1인 차원 제거
t7 = t6.squeeze() # Shape becomes (3,)

# dim 0 제거
t8 = t6.squeeze(0) # Shape becomes (3, 1)
print(t7)
print(t8)

tensor([1, 2, 3])
tensor([[1],
        [2],
        [3]])
```

```
In [94]: # Original tensor with shape (3,)
t9 = torch.tensor([1, 2, 3])

# dim 1 삽입
t10 = t9.unsqueeze(1) # Shape becomes (3, 1)
print(t10)

tensor([[1],
        [2],
        [3]])
```

```
In [95]: t11 = torch.tensor(
    [[1, 2, 3],
     [4, 5, 6]]
) # (2, 3)

# dim 1 삽입
t12 = t11.unsqueeze(1) # Shape becomes (2, 1, 3)
print(t12, t12.shape)

tensor([[[1, 2, 3],
         [4, 5, 6]]]) torch.Size([2, 1, 3])
```

```
In [96]: # Original tensor with shape (2, 3)
t13 = torch.tensor([1, 2, 3], [4, 5, 6])

# Flatten the tensor
t14 = t13.flatten() # Shape becomes (6,)

print(t14)

tensor([1, 2, 3, 4, 5, 6])
```

```
In [97]: # Original tensor with shape (2, 2, 2)
t15 = torch.tensor([[[1, 2],
                     [3, 4]],
                    [[5, 6],
                     [7, 8]]])

#! dim 0부터 flatten
t16 = torch.flatten(t15) # (8, )

#! dim 1부터 flatten
t17 = torch.flatten(t15, start_dim=1) # (2, 4)

print(t16)
print(t17)

tensor([1, 2, 3, 4, 5, 6, 7, 8])
tensor([[1, 2, 3, 4],
        [5, 6, 7, 8]])
```

```
In [98]: t18 = torch.randn(2, 3, 5)
print(t18.shape)

#! dim 0, 1, 2 축 순서 변경
print(torch.permute(t18, (2, 0, 1)).shape) # (2, 3, 5) -> (5, 2, 3)
```

```
torch.Size([2, 3, 5])
torch.Size([5, 2, 3])
```

```
In [99]: # Original tensor with shape (2, 3)
t19 = torch.tensor([[1, 2, 3], [4, 5, 6]])

# Permute the dimensions
t20 = torch.permute(t19, dims=(0, 1)) # Shape becomes (2, 3) still
t21 = torch.permute(t19, dims=(1, 0)) # Shape becomes (3, 2)
print(t20)
print(t21)
```

```
tensor([[1, 2, 3],
        [4, 5, 6]])
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

```
In [100... #! 두 차원의 순서 변경
t22 = torch.transpose(t19, 0, 1) # Shape becomes (3, 2)

print(t22)
```

```
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

```
In [101... #! 2차원 텐서 전치(transpose)
t23 = torch.t(t19) # Shape becomes (3, 2)

print(t23)
```

```
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

```
In [102... #! [추가 코드] CNN feature map을 batch 단위로 flatten
feature_maps = torch.randn(32, 64, 7, 7)
flattened = feature_maps.flatten(start_dim=1)

print(flattened.shape) # >>> torch.Size([32, 3136])
```

```
torch.Size([32, 3136])
```

```
In [103... #! [추가 코드] 단일 이미지에 batch 차원 추가
single_image = torch.randn(3, 224, 224)
batched_image = single_image.unsqueeze(0)

print(batched_image.shape) # >>> torch.Size([1, 3, 224, 224])
```

```
torch.Size([1, 3, 224, 224])
```

기술적 사항 및 고찰

- `view()` 와 `reshape()` 는 원소 수를 유지하며 shape을 바꾸고, `view()` 는 복사 없이 표현할 수 있을 때만 사용할 수 있음
- `squeeze()` 는 크기가 1인 차원을 제거하고 `unsqueeze()` 는 지정한 위치에 크기가 1인 차원을 추가함
- `flatten()` 은 여러 차원을 합치고, `permute()` 와 `transpose()` 는 원소를 다시 묶는 대신 축의 순서를 변경함

l_tensor_concat.py

```
In [104... import torch
```

```
In [105... t1 = torch.zeros([2, 1, 3])
t2 = torch.zeros([2, 3, 3])
t3 = torch.zeros([2, 2, 3])

#! dim 1을 기준으로 concatenate
t4 = torch.cat([t1, t2, t3], dim=1)
print(t4.shape)
```

```
torch.Size([2, 6, 3])
```

```
In [106... t5 = torch.arange(0, 3) # tensor([0, 1, 2])
t6 = torch.arange(3, 8) # tensor([3, 4, 5, 6, 7])

#! dim 0을 기준으로 concatenate
t7 = torch.cat((t5, t6), dim=0)
print(t7.shape) # >>> torch.Size([8])
print(t7) # >>> tensor([0, 1, 2, 3, 4, 5, 6, 7])
```

```
torch.Size([8])
tensor([0, 1, 2, 3, 4, 5, 6, 7])
```

```
In [107... t8 = torch.arange(0, 6).reshape(2, 3) # torch.Size([2, 3])
t9 = torch.arange(6, 12).reshape(2, 3) # torch.Size([2, 3])
```

```
#! 행 기준으로 병합
t10 = torch.cat((t8, t9), dim=0)
print(t10.size()) # >>> torch.Size([4, 3])
print(t10)
# >>> tensor([[ 0,  1,  2],
#              [ 3,  4,  5],
#              [ 6,  7,  8],
#              [ 9, 10, 11]])
```

```
torch.Size([4, 3])
tensor([[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8],
        [ 9, 10, 11]])
```

```
In [108... #! 열 기준으로 병합
t11 = torch.cat((t8, t9), dim=1)
print(t11.size()) # >>> torch.Size([2, 6])
print(t11)
# >>> tensor([[ 0,  1,  2,  6,  7,  8],
#              [ 3,  4,  5,  9, 10, 11]])
```

```
torch.Size([2, 6])
tensor([[ 0,  1,  2,  6,  7,  8],
        [ 3,  4,  5,  9, 10, 11]])
```

```
In [109... t12 = torch.arange(0, 6).reshape(2, 3) # torch.Size([2, 3])
t13 = torch.arange(6, 12).reshape(2, 3) # torch.Size([2, 3])
t14 = torch.arange(12, 18).reshape(2, 3) # torch.Size([2, 3])
```

```
#! 행렬 3개를 dim 0을 기준으로 concatenate
t15 = torch.cat((t12, t13, t14), dim=0)
print(t15.size()) # >>> torch.Size([6, 3])
print(t15)
# >>> tensor([[ 0,  1,  2],
#              [ 3,  4,  5],
#              [ 6,  7,  8],
#              [ 9, 10, 11],
#              [12, 13, 14],
#              [15, 16, 17]])
```

```
torch.Size([6, 3])
tensor([[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8],
        [ 9, 10, 11],
        [12, 13, 14],
        [15, 16, 17]])
```

```
In [110... #! 행렬 3개를 dim 1을 기준으로 concatenate
t16 = torch.cat((t12, t13, t14), dim=1)
print(t16.size()) # >>> torch.Size([2, 9])
print(t16)
# >>> tensor([[ 0,  1,  2,  6,  7,  8, 12, 13, 14],
#              [ 3,  4,  5,  9, 10, 11, 15, 16, 17]])
```

```
torch.Size([2, 9])
tensor([[ 0,  1,  2,  6,  7,  8, 12, 13, 14],
        [ 3,  4,  5,  9, 10, 11, 15, 16, 17]])
```

```
In [111... t17 = torch.arange(0, 6).reshape(1, 2, 3) # torch.Size([1, 2, 3])
t18 = torch.arange(6, 12).reshape(1, 2, 3) # torch.Size([1, 2, 3])
```

```
#! 3차원 텐서 2개를 dim 0을 기준으로 concatenate
t19 = torch.cat((t17, t18), dim=0)
print(t19.size()) # >>> torch.Size([2, 2, 3])
print(t19)
# >>> tensor([[[ 0,  1,  2],
#                [ 3,  4,  5]],
#              [[ 6,  7,  8],
#                [ 9, 10, 11]])
```

```
torch.Size([2, 2, 3])
tensor([[[ 0,  1,  2],
         [ 3,  4,  5]],
        [[ 6,  7,  8],
         [ 9, 10, 11]])
```

```
In [112... #! 3차원 텐서 2개를 dim 1을 기준으로 concatenate
t20 = torch.cat((t17, t18), dim=1)
```

```
print(t20.size()) #>>> torch.Size([1, 4, 3])
print(t20)
# >>> tensor([[[ 0,  1,  2],
#               [ 3,  4,  5],
#               [ 6,  7,  8],
#               [ 9, 10, 11]])

torch.Size([1, 4, 3])
tensor([[[ 0,  1,  2],
         [ 3,  4,  5],
         [ 6,  7,  8],
         [ 9, 10, 11]])])
```

```
In [113... #! 3차원 텐서 2개를 dim 2을 기준으로 concatenate

t21 = torch.cat((t17, t18), dim=2)
print(t21.size()) # >>> torch.Size([1, 2, 6])
print(t21)
# >>> tensor([[[ 0,  1,  2,  6,  7,  8],
#               [ 3,  4,  5,  9, 10, 11]])

torch.Size([1, 2, 6])
tensor([[[ 0,  1,  2,  6,  7,  8],
         [ 3,  4,  5,  9, 10, 11]])])
```

```
In [114... #! [추가 코드] 이미지 feature와 텍스트 feature 결합
image_features = torch.randn(32, 128)
text_features = torch.randn(32, 256)
combined_features = torch.cat([image_features, text_features], dim=1)

print(combined_features.shape) # >>> torch.Size([32, 384])

torch.Size([32, 384])
```

기술적 사항 및 고찰

- `cat()` 은 새로운 차원을 만들지 않고 지정한 기존 차원을 따라 Tensor들을 연결함
- 연결하는 차원의 결과 크기는 각 입력 크기의 합이며, 그 밖의 차원 크기는 모두 같아야 함
- (32, 128) 과 (32, 256) 을 `dim=1` 로 연결하면 batch 크기 32는 유지되고 feature 크기는 384가 됨

m_tensor_stacking.py

```
In [115... import torch

In [116... t1 = torch.tensor([[1, 2, 3], [4, 5, 6]]) #! (2, 3)
t2 = torch.tensor([[7, 8, 9], [10, 11, 12]]) #! (2, 3)

#! torch.stack(): 새로운 차원을 생성하여 텐서를 쌓음
t3 = torch.stack([t1, t2], dim=0) #! (2, 3) -> (2, 2, 3)
t4 = torch.cat([t1.unsqueeze(dim=0), t2.unsqueeze(dim=0)], dim=0) #! (2, 3) -> (2, 2, 3)
print(t3, t3.shape, t3.equal(t4))

tensor([[[ 1,  2,  3],
         [ 4,  5,  6]],

        [[ 7,  8,  9],
         [10, 11, 12]]) torch.Size([2, 2, 3]) True
```

```
In [117... #! dim 1을 기준으로 stack
t5 = torch.stack([t1, t2], dim=1) #! (2, 3) -> (2, 2, 3)
t6 = torch.cat([t1.unsqueeze(dim=1), t2.unsqueeze(dim=1)], dim=1) #! (2, 3) -> (2, 2, 3)
print(t5, t5.shape, t5.equal(t6))

tensor([[[ 1,  2,  3],
         [ 7,  8,  9]],

        [[ 4,  5,  6],
         [10, 11, 12]]) torch.Size([2, 2, 3]) True
torch.Size([2, 2, 3]) True
```

```
In [118... #! dim 2을 기준으로 stack
t7 = torch.stack([t1, t2], dim=2)
t8 = torch.cat([t1.unsqueeze(dim=2), t2.unsqueeze(dim=2)], dim=2)
print(t7, t7.shape, t7.equal(t8))

tensor([[[[ 1,  7],
          [ 2,  8],
          [ 3,  9]],

         [[ 4, 10],
          [ 5, 11],
          [ 6, 12]]]]) torch.Size([2, 3, 2]) True
```

```
In [119... t9 = torch.arange(0, 3) # tensor([0, 1, 2])
t10 = torch.arange(3, 6) # tensor([3, 4, 5])

print(t9.size(), t10.size())
# >>> torch.Size([3]) torch.Size([3])
```

torch.Size([3]) torch.Size([3])

```
In [120... t11 = torch.stack((t9, t10), dim=0)
print(t11.size()) # >>> torch.Size([2,3])
print(t11)
# >>> tensor([[0, 1, 2],
#             [3, 4, 5]])

t12 = torch.cat((t9.unsqueeze(0), t10.unsqueeze(0)), dim=0)
print(t11.equal(t12))
# >>> True
```

torch.Size([2, 3])
tensor([[0, 1, 2],
 [3, 4, 5]])

True
tensor([[0, 1, 2],
 [3, 4, 5]])

True

```
In [121... t13 = torch.stack((t9, t10), dim=1)
print(t13.size()) # >>> torch.Size([3,2])
print(t13)
# >>> tensor([[0, 3],
#             [1, 4],
#             [2, 5]])
t14 = torch.cat((t9.unsqueeze(1), t10.unsqueeze(1)), dim=1)
print(t13.equal(t14))
# >>> True
```

torch.Size([3, 2])
tensor([[0, 3],
 [1, 4],
 [2, 5]])

True

```
In [122... #! [추가 코드] 서로 다른 shape의 텐서를 stack하려고 하면 RuntimeError 발생
try:
    t15 = torch.arange(0, 6).reshape(2, 3)
    t16 = torch.arange(6, 14).reshape(2, 4)
    t17 = torch.stack((t15, t16), dim=0) #! RuntimeError
    print(t17.size())
except RuntimeError as error:
    print("서로 다른 shape의 텐서를 stack하려고 하면 RuntimeError 발생")
    print(type(error).__name__, error)
```

서로 다른 shape의 텐서를 stack하려고 하면 RuntimeError 발생

RuntimeError stack expects each tensor to be equal size, but got [2, 3] at entry 0 and [2, 4] at entry 1

```
In [123... #! [추가 코드] 여러 이미지 sample을 하나의 batch로 구성
sample1 = torch.randn(3, 224, 224)
sample2 = torch.randn(3, 224, 224)
batch = torch.stack([sample1, sample2], dim=0)

print(batch.shape) # >>> torch.Size([2, 3, 224, 224])
```

torch.Size([2, 3, 224, 224])

기술적 사항 및 고찰

- `stack()` 은 입력 Tensor를 쌓으면서 지정한 위치에 새로운 차원을 생성함
- 모든 입력 Tensor의 shape이 완전히 같아야 하며 broadcasting으로 크기를 맞춰주지 않음
- `stack(..., dim=d)` 는 각 입력에 `unsqueeze(d)` 를 적용한 뒤 `cat(..., dim=d)` 를 수행한 것과 같은 결과를 만듦

n_tensor_vstack_hstack.py

```
In [124... import torch
```

```
In [125... t1 = torch.tensor([1, 2, 3]) # (3, )
t2 = torch.tensor([4, 5, 6]) # (3, )

#! torch.vstack(): 1차원의 경우 unsqueeze(0)하여 dim=0을 기준으로 concat, 2차원 이상의 경우 dim=0 기준으로 concat
t3 = torch.vstack((t1, t2)) # (2, 3)
print(t3)
print(torch.cat((t1.unsqueeze(0), t2.unsqueeze(0)), dim=0).equal(t3))
```

```
tensor([[1, 2, 3],
        [4, 5, 6]])
```

True

```
In [126... t4 = torch.tensor([[1], [2], [3]]) # (3, 1)
t5 = torch.tensor([[4], [5], [6]]) # (3, 1)
t6 = torch.vstack((t4, t5))         # (6, 1)
print(t6)
print(torch.cat((t4, t5), dim=0).equal(t6))
```

```
tensor([[1],
        [2],
        [3],
        [4],
        [5],
        [6]])
```

True

```
In [127... t7 = torch.tensor([
    [[1, 2, 3], [4, 5, 6]],
    [[7, 8, 9], [10, 11, 12]]
])
t8 = torch.tensor([
    [[13, 14, 15], [16, 17, 18]],
    [[19, 20, 21], [22, 23, 24]]
])
t9 = torch.vstack([t7, t8])

print(t7, t7.shape) # (2, 2, 3)
print(t8, t8.shape) # (2, 2, 3)
print(t9, t9.shape) # (4, 2, 3)
```

```
tensor([[[ 1,  2,  3],
         [ 4,  5,  6]],
        [[ 7,  8,  9],
         [10, 11, 12]]]) torch.Size([2, 2, 3])
tensor([[[13, 14, 15],
         [16, 17, 18]],
        [[19, 20, 21],
         [22, 23, 24]]]) torch.Size([2, 2, 3])
tensor([[[ 1,  2,  3],
         [ 4,  5,  6]],
        [[ 7,  8,  9],
         [10, 11, 12]],
        [[13, 14, 15],
         [16, 17, 18]],
        [[19, 20, 21],
         [22, 23, 24]]]) torch.Size([4, 2, 3])
```

```
In [128... t10 = torch.tensor([1, 2, 3])  #! (3, )
t11 = torch.tensor([4, 5, 6])  #! (3, )

#! torch.hstack(): 1차원에서는 dim=0, 2차원 이상에서는 dim=1 기준으로 concatenate
t12 = torch.hstack((t10, t11))  #! (6, )
print(t12, t12.shape)

print(torch.cat((t10, t11), dim=0).equal(t12))
```

```
tensor([1, 2, 3, 4, 5, 6]) torch.Size([6])
True
```

```
In [129... t13 = torch.tensor([[1], [2], [3]]) #! (3, 1)
t14 = torch.tensor([[4], [5], [6]]) #! (3, 1)
t15 = torch.hstack((t13, t14))       #! (3, 2)
print(t15, t15.shape)

print(torch.cat((t13, t14), dim=1).equal(t15))
```

```
tensor([[1, 4],
        [2, 5],
        [3, 6]]) torch.Size([3, 2])
```

True

```
In [130... t16 = torch.tensor([
    [[1, 2, 3], [4, 5, 6]],
    [[7, 8, 9], [10, 11, 12]]
])
t17 = torch.tensor([
    [[13, 14, 15], [16, 17, 18]],
```

```

[[19, 20, 21], [22, 23, 24]]
])
print(t16.shape) # (2, 2, 3)
print(t17.shape) # (2, 2, 3)

t18 = torch.hstack([t16, t17])
print(t18.shape) # (2, 4, 3)
print(t18)

```

```

torch.Size([2, 2, 3])
torch.Size([2, 2, 3])
torch.Size([2, 4, 3])
tensor([[[ 1,  2,  3],
          [ 4,  5,  6],
          [13, 14, 15],
          [16, 17, 18]],

        [[ 7,  8,  9],
          [10, 11, 12],
          [19, 20, 21],
          [22, 23, 24]]])

```

기술적 사항 및 고찰

- `vstack()` 은 1차원 Tensor를 행으로 바꾸어 세로로 쌓고, 2차원 이상에서는 `dim=0` 방향으로 연결함
- `hstack()` 은 1차원에서는 `dim=0`, 2차원 이상에서는 `dim=1` 방향으로 연결함
- 두 함수도 broadcasting을 수행하지 않으므로 연결하지 않는 차원의 크기가 서로 맞아야 함

숙제 후기

벡터와 2차원 행렬 연산에는 익숙했지만, Tensor가 3차원 이상이 되면서 각 축이 무엇을 의미하는지 파악하는 과정이 어려웠다. 특히 broadcasting은 두 shape을 오른쪽부터 비교하고 크기가 1인 차원이 확장된다는 규칙을 직접 여러 예제에 적용하면서 이해할 수 있었다. `cat()` 은 기존 차원으로 연결하고 `stack()` 은 새로운 차원을 만든다는 차이도 결과 shape을 출력해 보면서 명확해졌다. 또한 `view()`, `reshape()`, `permute()` 는 모두 모양과 관련된 함수이지만 메모리 공유 여부와 축을 다루는 방식이 서로 다르다는 점을 알게 되었다. 앞으로는 연산 전에 각 Tensor의 shape과 dtype을 먼저 확인하는 습관을 들여야겠다고 생각했다.